

# Demystifying Agentic AI

Tool Calling, Skills, and Modern Architecture

---

Kyle Pounder    Toby Shearman

IAB Fall Meeting  
University of Arizona

# The task

**“The test suite is failing. Fix it.”**

What `run_tests()` shows (excerpt):

```
def test_median_even():
>     assert median([4, 1, 3, 2]) == 2.5
E     assert 3 == 2.5
E         + where 3 = median([4, 1, 3, 2])

FAILED test_stats.py::test_median_even
1 failed, 2 passed in 0.01s
```

The code under test, `stats.py`:

```
def mean(xs):
    return sum(xs) / len(xs)

def median(xs):
    xs = sorted(xs)
    n = len(xs)
    return xs[n // 2]
```

# A coding agent at work

> The test suite is failing. Fix it.

- I'll start by running the tests.

- `run_tests()`

```
FAILED test_stats.py::test_median_even
```

```
assert 3 == 2.5 (median([4, 1, 3, 2]) is 3)
```

```
1 failed, 2 passed
```

# A coding agent at work

✓ `run_tests()` 1 failed, 2 passed

- The failure is in `median`. Let me read it.

- `read_file("stats.py")`

```
def median(xs):  
    xs = sorted(xs)  
    n = len(xs)  
    return xs[n // 2]
```

# A coding agent at work

✓ `run_tests()` 1 failed, 2 passed

✓ `read_file("stats.py")`

- For even lengths it should average the middle two.

- `edit_file(`

- `path: "stats.py",`

- `old_text: "return xs[n//2]",`

- `new_text: "if n % 2 == 0:`

- `return (xs[n // 2 - 1] + xs[n // 2]) / 2`

- `return xs[n // 2]"`

`error: old_text not found in stats.py`

# A coding agent at work

✓ `run_tests()` 1 failed, 2 passed

✓ `read_file("stats.py")`

✗ `edit_file(...)` error: old\_text not found

- The spacing didn't match. Retrying with the exact text.

- `edit_file(`

- `path: "stats.py",`

- `old_text: "return xs[n // 2]",`

- `new_text: "if n % 2 == 0: ... return xs[n // 2]"`

- `ok`

# A coding agent at work

```
✓ run_tests() 1 failed, 2 passed
✓ read_file("stats.py")
✗ edit_file(...) error: old_text not found
✓ edit_file(...) ok
```

- `run_tests()`  
3 passed
- Fixed: median now averages the two middle values for even-length lists. All 3 tests pass.

# That looks like magic. What is going on?

## An LLM

chooses the  
next action  
and constructs  
its arguments

## Three tools

read\_file,  
run\_tests, edit\_file

## A loop

runs the action,  
adds the result to  
the history, repeats

Next: each piece, with the math that pins it down.

# Tools

---

# What is a tool? An example first: `read_file`

`read_file` is a function  $f$  that takes a path (given as JSON) as input and returns a string:

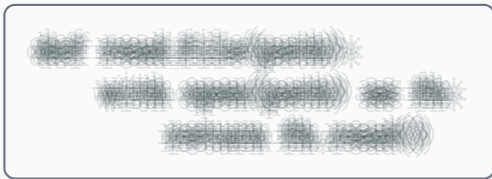
$$f : \{ \text{"path": string} \} \longrightarrow \text{string}$$
$$f(\{ \text{"path": "stats.py"} \}) = \text{"def median(xs): ..."}$$

# What is a tool? An example first: read\_file

`read_file` is a function  $f$  that takes a path (given as JSON) as input and returns a string:

$$f : \{ \text{"path": string} \} \longrightarrow \text{string}$$

$$f(\{ \text{"path": "stats.py"} \}) = \text{"def median(xs): ..."}"$$



The LLM does not see (and does not care to see) how  $f$  is defined.

# What is a tool?

A tool is an “interface” to a function, i.e.,

$$\tau = \left( \underbrace{\text{name}, S}_{\text{interface} \text{ — sent to the model}}, \underbrace{f}_{\text{implementation} \text{ — host-only}} \right)$$

- The **interface** (name,  $S$ ) is what the model reads and calls against. The tool has a name and  $S$  is the *schema* (usually a JSON Schema).
- The **implementation**  $f : S \rightarrow \mathcal{Y}$  is what runs, on the host, and is never sent to the model.
- $\mathcal{X}$ : all possible arguments ( $S \subseteq \mathcal{X}$ , the valid ones).  $\mathcal{Y}$ : all possible results.

The model writes down a call from the interface alone. The host checks  $x \in S$ , then runs  $f$ . If  $x \notin S$ , it returns an error message instead.

# Let's fill it in: $\tau = \text{edit\_file}$

**In plain English:** “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

|               |                      |
|---------------|----------------------|
| $S$           | <input type="text"/> |
| $f$           | <input type="text"/> |
|               | <input type="text"/> |
| $\mathcal{Y}$ | <input type="text"/> |

# Let's fill it in: $\tau = \text{edit\_file}$

**In plain English:** “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

$S$     objects with exactly the keys `path`, `old_text`, `new_text`

$f$    

$\gamma$

# Let's fill it in: $\tau = \text{edit\_file}$

**In plain English:** “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

$S$     objects with exactly the keys `path`, `old_text`, `new_text`

$f$     replace `old_text` by `new_text` in the file; return “ok” or an error message

$\gamma$     

# Let's fill it in: $\tau = \text{edit\_file}$

**In plain English:** “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

$S$     objects with exactly the keys `path`, `old_text`, `new_text`

$f$     replace `old_text` by `new_text` in the file; return “ok” or an error message

$\mathcal{V}$     string

# Tools in the example

| name      | <i>S</i> : allowed arguments       | <i>f</i> : what the host does | result in $\mathcal{V}$      |
|-----------|------------------------------------|-------------------------------|------------------------------|
| read_file | a path (string)                    | reads the file                | its contents (a string)      |
| run_tests | none, or a path                    | runs the test suite           | pass/fail summary, traceback |
| edit_file | a path, the old text, the new text | replaces old with new         | “ok” or an error message     |

# The Loop

---

# So we have tools. Now what?

How do we build an agent around that?

```
[
  {"name": "read_file",
   "description": "Read a file.",
   "input": {"path": "string"}},
  {"name": "run_tests",
   "description": "Run the test suite.",
   "input": {"path?": "string"}},
  {"name": "edit_file",
   "description": "Replace text in a file.",
   "input": {"path": "string",
             "old_text": "string",
             "new_text": "string"}},
  ...
]
```

# So we have tools. Now what?

How do we build an agent around that?

Build a loop!

At every step it needs to answer two questions:

What do I know?

What do I do?

```
[  
  {"name": "read_file",  
    "description": "Read a file.",  
    "input": {"path": "string"}},  
  {"name": "run_tests",  
    "description": "Run the test suite.",  
    "input": {"path?": "string"}},  
  {"name": "edit_file",  
    "description": "Replace text in a file.",  
    "input": {"path": "string",  
              "old_text": "string",  
              "new_text": "string"}},  
  ...  
]
```

# What do I know?

What the model receives on its next turn:

```
system  You are a coding agent working in this repository.  ...
tools   read_file(path) --- Read a file and return its contents.
        run_tests(path?) --- Run the test suite; return a summary.
        edit_file(path, old_text, new_text) --- Replace old_text with new_text.
user    The test suite is failing.  Fix it.
model   call(run_tests, {})
result  FAILED test_median_even:  assert 3 == 2.5.  1 failed, 2 passed
model   call(read_file, {path:  "stats.py"})
result  def median(xs):  xs = sorted(xs); n = len(xs); return xs[n // 2] ...
model   ← writes its next action here
```

Each model turn is either a `call` (run a tool, keep going) or an `answer` (reply to the user, stop).

# What do I know?

**Context:** a history of instructions, actions, and results. It starts as

$$h_0 = (\text{system prompt}, \{(\text{name}_\tau, S_\tau)\}_{\tau \in T}, \text{user request})$$

and each step appends one (action, result) pair:

$$h_{t+1} = h_t \frown (a_t, o_t), \quad h_t = h_0 \frown (a_0, o_0) \frown \cdots \frown (a_{t-1}, o_{t-1}) \in H$$

$a_t$ : the model's action at step  $t$ .  $o_t$ : the tool's output for that action.  $\frown$  appends to the end.

$H$ : all possible histories.

# What do I know?

**Context:** a history of instructions, actions, and results. It starts as

$$h_0 = (\text{system prompt}, \{(\text{name}_\tau, S_\tau)\}_{\tau \in T}, \text{user request})$$

and each step appends one (action, result) pair:

$$h_{t+1} = h_t \frown (a_t, o_t), \quad h_t = h_0 \frown (a_0, o_0) \frown \cdots \frown (a_{t-1}, o_{t-1}) \in H$$

$a_t$ : the model's action at step  $t$ .  $o_t$ : the tool's output for that action.  $\frown$  appends to the end.

$H$ : all possible histories.

**Conditioning:** the model's next choice depends on  $h_t$  and nothing else. If it isn't in  $h_t$ , the model doesn't know it.

# What do I do?

**Tools:** `read_file`, `run_tests`, `edit_file`, ..., **or** answer the user.

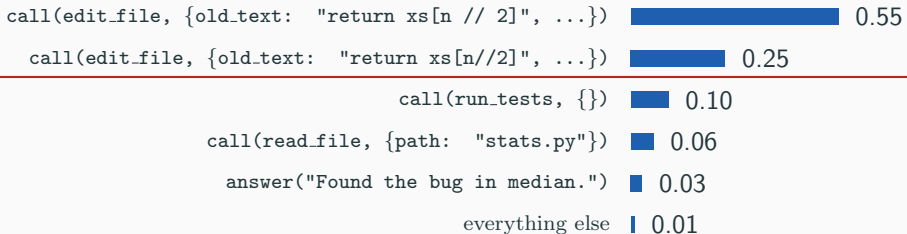
Probability of each next action, given the context  $h_2$  from before

|                                                                    |                                                                                     |      |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------|------|
| <code>call(edit_file, {old_text: "return xs[n // 2]", ...})</code> |  | 0.55 |
| <code>call(edit_file, {old_text: "return xs[n//2]", ...})</code>   |  | 0.25 |
| <code>call(run_tests, {})</code>                                   |  | 0.10 |
| <code>call(read_file, {path: "stats.py"})</code>                   |  | 0.06 |
| <code>answer("Found the bug in median.")</code>                    |  | 0.03 |
| everything else                                                    |  | 0.01 |

# What do I do?

**Tools:** `read_file`, `run_tests`, `edit_file`, ..., **or** answer the user.

Probability of each next action, given the context  $h_2$  from before



Same tool, different arguments: different actions. Choosing the tool and writing its arguments is one choice.

# What do I do?

An **action** is one of two kinds:

$$A = \{\text{answer}(m) : m \text{ a message}\} \cup \{\text{call}(\tau, x) : \tau \in T, x \in \mathcal{X}\}$$

$\text{call}(\tau, x)$     ask the host to run tool  $\tau$  on arguments  $x$ ; the loop continues

$\text{answer}(m)$     reply to the user with the text  $m$ ; the loop stops

# What do I do?

An **action** is one of two kinds:

$$A = \{\text{answer}(m) : m \text{ a message}\} \cup \{\text{call}(\tau, x) : \tau \in T, x \in \mathcal{X}\}$$

$\text{call}(\tau, x)$  ask the host to run tool  $\tau$  on arguments  $x$ ; the loop continues

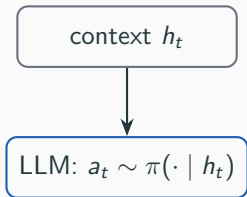
$\text{answer}(m)$  reply to the user with the text  $m$ ; the loop stops

The **policy** is the LLM:

$$\pi : H \rightarrow \Delta(A), \quad a_t \sim \pi(\cdot \mid h_t)$$

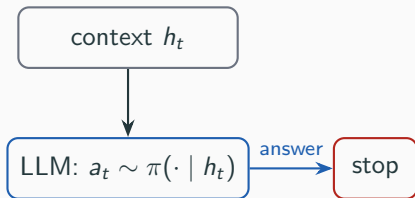
$\Delta(A)$ : probability distributions on  $A$ . Note  $x \in \mathcal{X}$ , not  $S_\tau$ : the model can write invalid arguments.  $\pi$  is fixed; we only sample from it.

# Putting it together



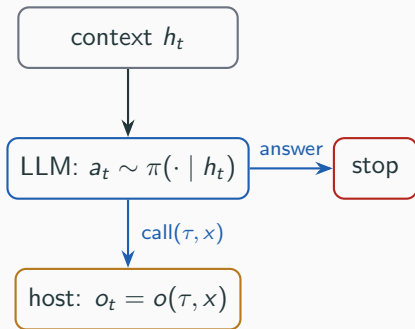
1. Sample an action:  $a_t \sim \pi(\cdot \mid h_t)$ .

# Putting it together



1. Sample an action:  $a_t \sim \pi(\cdot \mid h_t)$ .
2. If  $a_t = \text{answer}(m)$ : stop and return  $m$ .

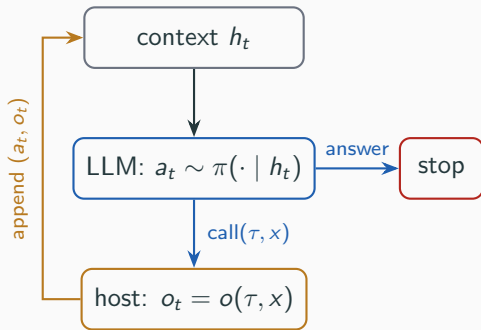
# Putting it together



1. Sample an action:  $a_t \sim \pi(\cdot \mid h_t)$ .
2. If  $a_t = \text{answer}(m)$ : stop and return  $m$ .
3. If  $a_t = \text{call}(\tau, x)$ : the host returns

$$o_t = \begin{cases} f_\tau(x) & x \in S_\tau \\ \text{an error message} & \text{otherwise} \end{cases}$$

# Putting it together

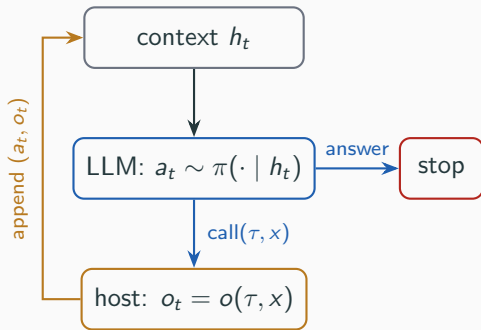


1. Sample an action:  $a_t \sim \pi(\cdot | h_t)$ .
2. If  $a_t = \text{answer}(m)$ : stop and return  $m$ .
3. If  $a_t = \text{call}(\tau, x)$ : the host returns

$$o_t = \begin{cases} f_\tau(x) & x \in S_\tau \\ \text{an error message} & \text{otherwise} \end{cases}$$

4. Append:  $h_{t+1} = h_t \frown (a_t, o_t)$ . Repeat.

# Putting it together



1. Sample an action:  $a_t \sim \pi(\cdot | h_t)$ .
2. If  $a_t = \text{answer}(m)$ : stop and return  $m$ .
3. If  $a_t = \text{call}(\tau, x)$ : the host returns

$$o_t = \begin{cases} f_\tau(x) & x \in S_\tau \\ \text{an error message} & \text{otherwise} \end{cases}$$

4. Append:  $h_{t+1} = h_t \frown (a_t, o_t)$ . Repeat.
5. Also stop when  $t = N$ , a step budget.

# The loop, on our example

$h_0$  = "The test suite is failing. Fix it." Each step appends one (action, result) pair. What does each step return?

| $t$ | action $a_t$                                                     | result $o_t$           |
|-----|------------------------------------------------------------------|------------------------|
| 0   | <code>call(run_tests, {})</code>                                 | 1 failed, 2 passed     |
| 1   | <code>call(read_file, {path: "stats.py"})</code>                 | the source of stats.py |
| 2   | <code>call(edit_file, {old_text: "return xs[n//2]", ...})</code> |                        |

Step 2's old\_text looks right – what actually happens?

# The loop, on our example

$h_0$  = "The test suite is failing. Fix it." Each step appends one (action, result) pair. What does each step return?

| $t$ | action $a_t$                                                     | result $o_t$              |
|-----|------------------------------------------------------------------|---------------------------|
| 0   | <code>call(run_tests, {})</code>                                 | 1 failed, 2 passed        |
| 1   | <code>call(read_file, {path: "stats.py"})</code>                 | the source of stats.py    |
| 2   | <code>call(edit_file, {old_text: "return xs[n//2]", ...})</code> | error: old_text not found |

$o_2$  is an error (the file has `xs[n // 2]`); it lands in  $h_3$ , and the loop continues.

# The loop, on our example

$h_0$  = "The test suite is failing. Fix it." Each step appends one (action, result) pair. What does each step return?

| $t$ | action $a_t$                                                       | result $o_t$              |
|-----|--------------------------------------------------------------------|---------------------------|
| 0   | <code>call(run_tests, {})</code>                                   | 1 failed, 2 passed        |
| 1   | <code>call(read_file, {path: "stats.py"})</code>                   | the source of stats.py    |
| 2   | <code>call(edit_file, {old_text: "return xs[n//2]", ...})</code>   | error: old_text not found |
| 3   | <code>call(edit_file, {old_text: "return xs[n // 2]", ...})</code> | ok                        |
| 4   | <code>call(run_tests, {})</code>                                   | 3 passed                  |
| 5   | <code>answer("Fixed median; 3 tests pass.")</code>                 |                           |

Step 5 is an answer. Does the loop take another step?

# The loop, on our example

$h_0$  = "The test suite is failing. Fix it." Each step appends one (action, result) pair. What does each step return?

| $t$ | action $a_t$                                                       | result $o_t$              |
|-----|--------------------------------------------------------------------|---------------------------|
| 0   | <code>call(run_tests, {})</code>                                   | 1 failed, 2 passed        |
| 1   | <code>call(read_file, {path: "stats.py"})</code>                   | the source of stats.py    |
| 2   | <code>call(edit_file, {old_text: "return xs[n//2]", ...})</code>   | error: old_text not found |
| 3   | <code>call(edit_file, {old_text: "return xs[n // 2]", ...})</code> | ok                        |
| 4   | <code>call(run_tests, {})</code>                                   | 3 passed                  |
| 5   | <code>answer("Fixed median; 3 tests pass.")</code>                 | (loop stops)              |

$a_5 = \text{answer}(m)$ , so the loop stops and returns  $m$ . There is no  $o_5$  and no  $h_6$ .

# Consequences

- **The context only grows.** Each step appends to  $h_t$ , but the model can only read a bounded amount. **Compaction** keeps  $h_0$  and the last  $k$  steps, and replaces the rest by a summary  $s$  written by the LLM:

$$h_t \mapsto h_0 \frown s \frown (a_{t-k}, o_{t-k}) \frown \cdots \frown (a_{t-1}, o_{t-1})$$

# Consequences

- **The context only grows.** Each step appends to  $h_t$ , but the model can only read a bounded amount. **Compaction** keeps  $h_0$  and the last  $k$  steps, and replaces the rest by a summary  $s$  written by the LLM:

$$h_t \mapsto h_0 \frown s \frown (a_{t-k}, o_{t-k}) \frown \cdots \frown (a_{t-1}, o_{t-1})$$

- **Tools are simply side-effectful functions.**  $f_\tau$  can do anything that ends in a result, including running another loop. A *sub-agent* is one tool call, e.g. `call(delegate, {task: "find every caller of median"})`:

$$f_{\text{delegate}}(x) = m \quad \text{where} \quad \underbrace{h'_0 = (\dots, x)}_{\text{a fresh loop}} \rightsquigarrow \text{answer}(m)$$

How do sub-agents affect the context?

# Skills

---

# What is a skill?

A skill is a set of written instructions the agent reads when a task calls for them:

$$\sigma = \left( \underbrace{\text{name, description}}_{\text{always in the history}}, \underbrace{\text{body}}_{\text{read only when needed}} \right)$$

- **Session start:** the host puts every skill's name and one-line description into  $h_0$ , next to the user's request.

# What is a skill?

A skill is a set of written instructions the agent reads when a task calls for them:

$$\sigma = \left( \underbrace{\text{name, description}}_{\text{always in the history}}, \underbrace{\text{body}}_{\text{read only when needed}} \right)$$

- **Session start:** the host puts every skill's name and one-line description into  $h_0$ , next to the user's request.
- **A task matches a description:** the model reads the body with an ordinary tool call,

$$h_{t+1} = h_t \frown (\text{call}(\text{read\_file}, \{\text{path} : \text{".../SKILL.md"}\}), \text{body})$$

# What is a skill?

A skill is a set of written instructions the agent reads when a task calls for them:

$$\sigma = \left( \underbrace{\text{name, description}}_{\text{always in the history}}, \underbrace{\text{body}}_{\text{read only when needed}} \right)$$

- **Session start:** the host puts every skill's name and one-line description into  $h_0$ , next to the user's request.
- **A task matches a description:** the model reads the body with an ordinary tool call,

$$h_{t+1} = h_t \frown (\text{call}(\text{read\_file}, \{\text{path} : \text{".../SKILL.md"}\}), \text{body})$$

- **From then on,** the instructions are part of what the model reads on every step. If they mention a bundled script, the model runs it the same way it runs any tool.

# A tool vs. a skill

|                 | <b>Tool</b> $\tau = (\text{name}, S, f)$          | <b>Skill</b> $\sigma = (\text{name}, \text{desc}, \text{body})$ |
|-----------------|---------------------------------------------------|-----------------------------------------------------------------|
| What it is      | a function the host runs                          | text the model reads                                            |
| Written in      | code                                              | plain language (Markdown)                                       |
| Seen up front   | name and schema $S$                               | name and description                                            |
| What it changes | the actions $A$ : adds $\text{call}(\tau, \cdot)$ | the history $h_t$ : what $\pi$ conditions on                    |
| Gives the agent | a new <i>capability</i>                           | <i>know-how</i> for using the ones it has                       |

A tool is a new instrument; a skill is the manual for a job. A skill adds no new actions: reading it is a `read_file` call.

# Example: a skill for our task

A folder with one Markdown file, plus optional scripts.

```
fix-failing-tests/  
  SKILL.md
```

**Name and description:** all the model sees until the task matches.

**Body:** read only after that.

```
---  
name:  fix-failing-tests  
description:  Use when asked to fix  
              failing tests or a failing build.  
---  
  
1.  Run the full test suite first; read  
    the failure before opening any file.  
2.  Read the code under test, not just  
    the test.  
3.  Make the smallest change that fixes  
    the failure.  
4.  Do not edit a test to make it pass  
    unless asked.  
5.  Rerun the full suite, not just the  
    failing test.  
6.  Report what you changed and why.
```

# Why skills: context budget

Why not put every instruction in  $h_0$ ? Because every token has a cost.

$n$  skills, each with a description of  $d$  tokens and a body of  $c$  tokens;  $k$  skills used:

$$\underbrace{n(d + c)}_{\text{load everything}} \quad \text{vs.} \quad \underbrace{nd + kc}_{\text{index, then load on demand}}$$

Illustrative numbers:  $n = 50$ ,  $d = 50$ ,  $c = 2000$ ,  $k = 1$  gives 102,500 versus 4,500 tokens.

The history is bounded (the model's context window), and instructions for unrelated tasks can distract the model. On demand, the agent can have hundreds of skills installed and pay for the one it uses.

# Why skills: precision

A skill moves probability toward the steps you want.

$\pi(\cdot \mid h)$  right after reading `stats.py` (made-up numbers)

■ without skill

fix median in `stats.py` 0.70

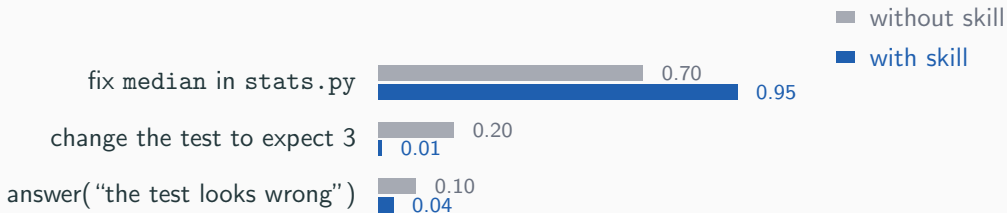
change the test to expect 3 0.20

answer("the test looks wrong") 0.10

# Why skills: precision

A skill moves probability toward the steps you want.

$\pi(\cdot \mid h)$  right after reading `stats.py` (made-up numbers)



Step 4 of the skill ("Do not edit a test to make it pass") removes a shortcut the model would otherwise sometimes take. The skill encodes *your* rules, which the model cannot infer from the code.

# Why skills: more reasons

- **Anyone can write one.** It's plain language: a domain expert can write down how a job is done without writing code or retraining the model.
- **Reusable and shareable.** A skill is a folder of files: version it, review it, and share it across a team and across agents.
- **Exact steps can be code.** A skill can bundle a script, so a step that must be exact (a format check, a calculation) is run by the host instead of improvised by the model.
- **Scoping.** A skill can also narrow which tools are allowed while it's active: fewer ways to be wrong, and least privilege.

# Takeaways

1. An agent is a model in a loop: it emits structured calls, the host validates and runs them, and the results go back into the history.
2. Reliability is a design problem: precise schemas, small scoped tool sets, and verification steps (like rerunning the tests) make errors easier to catch.

**Questions?**